Supplementary Material for "PORTER: A Plan-Committed Router for Multi-Core Distributed Quantum Computers"

This supplement contains Appendices A–K of the main paper. Section, figure, and table numbers of the form "Section III-A" or "Table VII" refer to the main paper; numbers local to this document refer to the supplement itself.

Appendix A
The 80-Qubit Suite: Reconstruction Detail

The 80-qubit suite (Section IV-B of the main paper) was generated with `mqt.bench` 2.2.2 [1] (device-independent generation followed by transpilation to our basis, since the installed `mqt.bench`'s target-aware native-gate path raised an unrelated `qiskit` compatibility error on every circuit we tried). AE at this size is not directly producible by stock `mqt.bench`: its phase-estimation construction evaluates $\mathrm{Operator}(Q)^{2^j}$ for each evaluation bit j, and the accumulated floating-point non-unitarity trips `qiskit`'s unitarity check (`"Input matrix is not unitary"`) beyond roughly 27 evaluation qubits, which is why published AE files stop near 54 qubits. Since the Bernoulli operator Q is a single-qubit rotation, we instead supply the power analytically, $Q^{2^j} = R_Y(2^{j+1}\theta_p)$ (R_Y periodic in 4π), and otherwise run `mqt.bench`'s own native-gate transpilation unchanged. The CNOT sub-sequence—which determines the two-qubit routing instance under the cost model used here—is decomposition-invariant in these rotation angles and is byte-identical to stock `mqt.bench` at every size the latter can build (verified at 10, 20, 25, and 28 qubits); only the routing-irrelevant single-qubit angles carry floating-point error at high bit indices.

QNN required an analogous reconstruction. Between `mqt.bench` 1.1.0 (which produced the deep 8,126-CX 64q QNN this suite uses) and the installed 2.2.2, the benchmark's definition was changed from a ZZ feature map (full all-pairs entanglement) plus a `RealAmplitudes` ansatz to a plain Z feature map (no entanglement) plus the same ansatz. The current generator therefore emits a near-linear 79-CX circuit at 80 qubits whose two-qubit interaction graph is *identical* to GHZ's — a degenerate duplicate, not a distinct benchmark. We restore the original deep QNN by rebuilding the 1.1.0 construction (ZZ feature map, `reps`=2, full entanglement; `RealAmplitudes`, `reps`=1) and transpiling it to our basis; at 64 qubits this reproduces the checked-in 1.1.0 file's CNOT sub-sequence byte-for-byte (identical ordered pair sequence, all $\binom{64}{2}$ interacting pairs, 8,126 CX), so the 80-qubit instance (a fully connected $\binom{80}{2}$-edge interaction graph with 12,718 CX gates) is a reproducible extrapolation of the same construction. Table I lists the CX count of every benchmark instance used in the paper.

TABLE I
BENCHMARK CIRCUIT CX COUNTS (MEASUREMENTS/BARRIERS STRIPPED).

Circuit	25q (arch. A)	64q (arch. H)	80q (arch. H)
AE	558	1962	2780
GHZ	24	63	79
Graphstate	25	64	80
QFT	580	1966	2660
QNN	1223	8126	12718
Random	1124	1627	23381

Appendix B
External Layouts from pytket-dqc

As a third, independent layout source complementing the shared-layout protocol of Section IV-C of the main paper, we route DSABRE and PORTER from a placement generated by `pytket-dqc`'s `HypergraphPartitioning` allocator [2] (best-of-3 KaHyPar seeds on a `NISQNetwork` matching our core graph), converted to a physical layout by the same topology-aware intra-core assignment DSABRE's own pytket benchmark uses [3]: each core's assigned logical qubits are placed by descending interaction-graph degree onto physical positions ordered by ascending intra-core centrality, then locally refined by a bounded pairwise-swap pass. TELESABRE is excluded for the same structural reason as in Section IV-C (no externally supplied layout support in its CLI). `HypergraphPartitioning` only decides which *core* each logical qubit goes to; the capacity it sees for a core is the length of that core's declared qubit list in the `NISQNetwork`. We withhold from every core's declared capacity, and from the later physical-placement step, the four minimum-degree (corner) qubits of the whole-chip graph — the chip-wide corner rule DSABRE shipped when these legs were run, rather than the per-core count its own protocol now uses (Section IV-C). As there, the reserved qubits hold spare capacity; they are not communication ports. This guarantees only a four-qubit margin *chip-wide*, not per-core, so a core can still end up fully packed; rather than skip the run in that case, PORTER's general safeguard against fully packed cores (Section III-A) legalizes the layout automatically before routing begins. DSABRE's teledata-only mechanism does not need a free port on the side it teleports *from* and was never affected by this precondition. Because `pytket-dqc` uses gate teleportation, not state teleportation, its own reported cost is in a different unit (e-bits per Steiner-tree-amortised hyperedge, not EPR pairs per qubit move) and is included only as context, following DSABRE's own established convention

TABLE II
PYTKET-MAPPING SHARED-LAYOUT COMPARISON OVER ALL TWENTY-FOUR INSTANCES, CORNERS-REMOVED NISQNETWORK CAPACITY. BOTH ROUTERS RUN THEIR OWN PROTOCOL FROM THE SAME PARTITION. PYTKET E-BITS IS HYPERGRAPHPARTITIONING'S OWN COST IN A DIFFERENT UNIT (CONTEXT ONLY, NOT EPR-COMPARABLE). PORTER'S GENERAL FULL-CORE SAFEGUARD LEAVES NO MISSING ENTRIES ON THIS SUITE.

Suite	Circuit	pytket e-bits	DSABRE		PORTER	
			EPR	SWAP	EPR	SWAP
25q	ae	78	68	282	73	348
25q	ghz	3	3	17	3	17
25q	graphstate	4	4	17	4	19
25q	qft	70	91	406	96	388
25q	qnn	148	151	714	127	703
25q	random	564	431	1409	395	1340
64q	ae	264	213	1530	190	1476
64q	ghz	5	5	58	5	62
64q	graphstate	10	8	53	10	61
64q	qft	307	273	1834	231	1700
64q	qnn	621	498	5165	800	6491
64q	random	1020	728	3704	787	4366
64q	qpeexact	214	220	1714	232	1799
64q	qaoa	492	576	3745	763	4750
64q	multiplier	3725	1828	10207	1322	9669
80q	ae	563	322	2536	385	2557
80q	ghz	7	7	83	7	86
80q	graphstate	10	11	86	10	77
80q	qft	502	389	2894	374	2377
80q	qnn	1045	827	8538	1408	11162
80q	random	20630	10642	57191	11115	61944
80q	qpeexact	218	307	2289	330	2406
80q	qaoa	777	899	6110	1143	7581
80q	multiplier	5682	6622	29964	2805	16764

for this comparison [3].

Table II reports a complete grid: PORTER's full-core safeguard legalizes every partition that would otherwise violate its teleport precondition, so neither router skips a row. DSABRE takes the grid on the geometric mean, 164.1 against PORTER's 167.2 (1.9%), using fewer EPR pairs on twelve of the twenty-four rows to PORTER's eight, with four ties. PORTER's worst relative row is 80q QNN (1,408 against 827).

The aggregate changes with the estimand. The instance-weighted geometric mean favours DSABRE, whereas the total EPR count favours PORTER: summed over the grid, PORTER spends 22,615 EPR pairs against DSABRE's 25,123 (10.0% fewer). This total is dominated by 80q Multiplier, on which PORTER uses 2,805 pairs against 6,622. Conversely, the unweighted geometric mean attenuates such high-cost rows. We therefore treat the geometric mean as the primary per-instance summary and report the total as a workload-weighted sensitivity analysis, rather than presenting either aggregate as a universal verdict. Pooled with the TELESABRE-layout grid of Table III of the main paper, DSABRE uses fewer EPR pairs on twenty of the forty-one fixed-layout instances, PORTER on sixteen, with five ties (Section V-B).

The grid above feeds pytket-dqc's partition to PORTER as a controlled probe. The complementary pipeline question – offering the same partition to PORTER as an optional extra layout candidate beside its own, which can lower EPR consumption on a small subset of instances but remains opt-in because of its cost and robustness trade-offs – is a configuration study of PORTER itself rather than part of this comparison, and is deferred to Appendix C.

APPENDIX C
PYTKET-DQC AS AN OPT-IN PORTER LAYOUT CANDIDATE

Appendix B feeds pytket-dqc's partition to PORTER as a controlled probe. The same partition can also be offered to PORTER as one extra layout candidate beside its own (Section III-A), routing both and keeping the lower EPR count. Conditional on both routes completing, selecting the lower-EPR route cannot increase EPR consumption. The relevant questions are therefore the size of that conditional gain, its preprocessing cost, and its robustness. Section V-B states the conclusion; this appendix gives the figures behind it.

What it gains. Over all twenty-four instances the best-of-two lowers geometric-mean EPR from 169.4 to 150.6 (-11.1%). Over the eighteen excluding GHZ and Graphstate it lowers it from 445.2 to 443.3 — $\mathbf{-0.4}\%$. The partition is selected on seven of the twenty-four, and the three largest margins are Graphstate at each width ($7 \to 4$, $20 \to 10$, $28 \to 10$); GHZ contributes $4 \to 3$, $6 \to 5$ and a tie. Everything else it wins, it wins narrowly. On the dense circuits it is the worse start and the best-of simply keeps PORTER's own layout (64q QNN 550 against 800, 64q Random 732 against 787, 80q AE 265 against 385).

What it costs. Table III times the best-of-three KaHyPar partition against PORTER's own layout construction and against PORTER's complete four-route protocol, on the same machine as every other measurement in this paper. The partition is three to five orders of magnitude dearer than the layout it would replace, and on seven of the eight circuits shown it costs more than routing the circuit four times — 64q Random the exception. It also scales worse than the routing does: a $42\times$ increase in CX count from 25q AE to 80q Random buys a $196\times$ increase in partition time, and at the widths of Section V-D it does not finish at all.

TABLE III
COST OF THE PYTKET-DQC PARTITION (BEST OF THREE KAHYPAR SEEDS) AGAINST PORTER'S OWN INITIAL-LAYOUT CONSTRUCTION AND AGAINST PORTER'S COMPLETE FOUR-ROUTE PROTOCOL. ONE MEASUREMENT EACH, SAME MACHINE AND CONDITIONS AS SECTION IV-B.

Suite	Circuit	CX	partition		PORTER
			time	vs. layout	4 routes
80q	graphstate	80	0.6 s	2,000×	0.5 s
25q	ae	558	3.2 s	10,225×	0.5 s
25q	random	1,124	7.3 s	11,968×	2.4 s
64q	random	1,627	11.9 s	11,856×	13.2 s
64q	ae	1,962	14.9 s	13,820×	1.8 s
80q	ae	2,780	20.5 s	13,521×	3.1 s
64q	qnn	8,126	110.8 s	35,309×	19.2 s
80q	random	23,381	626.9 s	64,184×	116.5 s

Why it stays opt-in. The gain is 0.4% once the two lowest-CX circuit families are set aside, and it is bought at

a preprocessing cost that exceeds the whole compilation. A third consideration is robustness: the partition reintroduces an external dependency that PORTER's self-contained layout avoids, and unlike it can leave a core fully packed, relying on the full-core repair of Section III-A to become routable at all. PORTER therefore defaults to its own layout and treats an external partition as an optional refinement, worth its price only where EPR is at a premium, the circuit is cleanly separable, and the partitioning time is acceptable.

APPENDIX D
INDEPENDENT REPLAY VERIFICATION

Beyond code review, we built an independent replay verifier that treats the router as a black box: it takes only the emitted `CompiledOp` stream, the original circuit, and the initial layout, and re-derives, from scratch, whether the stream is a faithful and physically legal execution. The verifier maintains its own physical-position map and a per-logical-qubit FIFO queue of pending gate IDs (derived directly from the circuit, independent of any router-internal DAG state), and checks, per emitted operation:

- every gate is popped from the head of *every* one of its operand qubits' queues before being marked executed, so gates cannot execute out of DAG order or twice;
- every `local_swap`/`local_cx` acts on a genuine intra-core coupling edge, at the physical positions the verifier's own simulated layout says the operand logical qubits occupy;
- every `teleport` uses the correct inter-core port pair for its two cores, both ports were free beforehand, and the source qubit was coupling-adjacent to the source port;
- every `cat_entangler`, `cat_cx`, and `cat_disentangler` triple observes port-adjacency and free-port preconditions, and no other operation touches either communication port while a cat section is open;
- the emitted `RouterStats` counters (EPR, teleport, telegate, packet, SWAP, local-CX, single counts) match the counts the verifier itself derives from the op stream, catching any bookkeeping drift between what is emitted and what is counted.

We ran the verifier against 60 randomized circuits (4–40 logical qubits, 20–600 gates, both architectures, seeds 0–59) and against every circuit in the evaluation suite (Section IV of the main paper); all pass. Two further artefacts make these checks concrete: a *certified trace* (Figure 1) and a *fault-injection study* (Table IV).

A certified trace. Figure 1 replays the stream PORTER emits for the running example of the main paper (Section III-D) — fourteen operations exercising every op type — and lists, per operation, exactly what the verifier asserts before accepting it. The trace makes the three primitives' legality conditions operational: the TELEGATEPACKET is one *bracketed section* in which the two reserved ports are untouchable and every interleaved SWAP must carry packet provenance; the TELEDATA at op 11 must name the unique port pair of its core pair, find both ports free, start port-adjacent, and land exactly on the destination port; and every gate execution, local or remote, is popped from the head of each operand's pending-gate queue, so DAG order is enforced gate by gate rather than assumed from the router's own bookkeeping.

#	emitted operation		verifier asserts
0	`local_swap` $p_3 \leftrightarrow p_4$		coupling edge; operands sit where the replayed layout says
1	`cat_entangler` q_0; (p_5, p_{12}); 1 EPR		no section already open; *exact* port pair of (c_0, c_1); both ports free; q_0 adjacent to p_5; opens section, reserving p_5, p_{12}
2	`local_swap` $p_{10} \leftrightarrow p_9$	route q_1	tagged as packet routing; touches neither reserved port
3	`cat_cx` g_1 (p_{12}, p_9)		g_1 at the head of *both* q_0's and q_1's queues; control = section control; proxy is the target-side port p_{12}; target in c_1, adjacent to p_{12}
4	`local_swap` $p_{13} \leftrightarrow p_{14}$	route q_2	as op 2
5	`cat_cx` g_2 (p_{12}, p_{13})		as op 3
6	`cat_cx` g_3 (p_{12}, p_{15})		as op 3 (q_3 already port-adjacent)
7	`cat_disentangler`		a section is open; closes it, releasing the ports
8	`local_swap` $p_6 \leftrightarrow p_7$		coupling edge
9	`local_cx` g_5 (p_4, p_7)		queue heads; same core; coupling edge
10	`local_swap` $p_{13} \leftrightarrow p_{14}$		coupling edge
11	`teleport` q_5: $p_{13} \rightarrow p_5$; 1 EPR		cores adjacent; exact port pair (p_{12}, p_5); both ports free; q_5 adjacent to p_{12}; lands exactly on p_5
12	`local_swap` $p_4 \leftrightarrow p_5$		coupling edge
13	`local_cx` g_4 (p_3, p_4)		queue heads; coupling edge

Fig. 1. The full operation stream PORTER emits for the running example of the main paper (Section III-D, Figure 4), as replayed by the verifier. Ops 1–7 execute inside the live cat section, where only packet-tagged SWAPs and 1Q gates that avoid the two reserved ports are legal. At end of stream the verifier further asserts: no cat section left open, all five gates executed exactly once, and the emitted statistics equal to the derived counts (2 EPR, 1 teleport, 1 packet, 0 single telegates, 6 SWAPs, 2 local CXs).

TABLE IV
FAULT-INJECTION VALIDATION OF THE VERIFIER (`VERIFY_FAULT_INJECTION.PY`): SINGLE-POINT CORRUPTIONS APPLIED TO OPERATION STREAMS THAT PASS VERIFICATION, EACH OF WHICH MUST BE REJECTED.

Injected corruption	Rejecting check
delete a `cat_disentangler`	section state machine
swap a teleport's two ports	exact inter-core port pair
zero a teleport's EPR cost	per-op EPR accounting
duplicate a gate-carrying op	every gate exactly once
delete a gate-carrying op	queue order / end-of-stream coverage
swap two ops sharing a qubit	per-qubit dependency order
retarget a SWAP to a non-coupled pair	coupling-edge legality
untag an in-section routing SWAP	cat-section isolation
retarget an in-section SWAP onto a port	reserved-port protection
increment `stats.epr` by one	stats–stream cross-check

Fault injection. On streams that always pass, a verifier that checks nothing is indistinguishable from one that checks everything, so we also certify that each check *fires*: `verify_fault_injection.py` takes passing op streams and applies the ten single-point corruptions of Table IV, one mutant per corruption, spanning misuse of all three communication primitives (wrong ports, free-port violations, broken cat-section bracketing), gate-coverage and dependency-order violations, illegal SWAPs, and statistics drift. Across the running example (which admits all ten corruption classes) and thirty fuzzed circuits on both architectures, all 276 applicable mutants are rejected, each by the intended check; the same

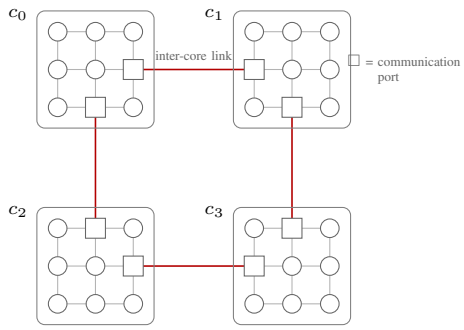


Fig. 2. Architecture A, introduced by TELESABRE [4]: a 2×2 array of cores, each a 3×3 grid of physical qubits (circles) with nearest-neighbour intra-core coupling (grey edges). Squares are communication ports; every inter-core link joins one port in each of two adjacent cores and is the only place EPR pairs are consumed. Architecture H (2×3 array of 4×4 cores, Section IV) follows the same pattern.

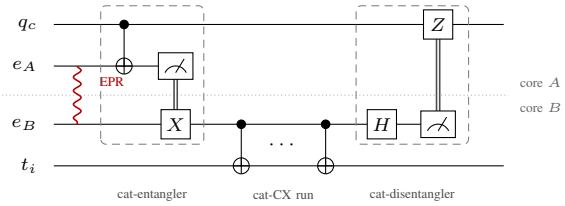


Fig. 3. The cat-entangler and cat-disentangler protocol [5] underlying TELEGATE and TELEGATEPACKET. The cat-entangler (left dashed box) consumes the EPR pair between ports e_A and e_B: $\mathrm{CX}(q_c, e_A)$, measurement of e_A, and a classically controlled X (double lines are classical bits) leave (q_c, e_B) in the cat state $\alpha|00\rangle + \beta|11\rangle$, making the remote port e_B a proxy for the control q_c. Every CX controlled by e_B (middle) then acts as if controlled by q_c — exactly one for TELEGATE, an ordered same-control run (interleaved with target-side 1Q gates) for TELEGATEPACKET. The cat-disentangler (right dashed box) — Hadamard, measurement, classically controlled Z — restores q_c and frees both ports.

harness runs on any benchmark QASM file.

APPENDIX E
ARCHITECTURE AND PRIMITIVE FIGURES

Figure 2 draws architecture A, the smaller of the two grid-of-grids machines of Section IV-B, and Figure 3 gives the cat-entangler and cat-disentangler constructions that TELEGATE and TELEGATEPACKET are built from.

APPENDIX F
RUNNING EXAMPLE: WHY COMMIT TO A PLAN?

Figure 4 shows a front where plan commitment, the packet vocabulary, and the $f_{\min}$ capacity gate all interact, and where PORTER's decision differs from what either baseline would choose. Two adjacent cores of architecture A are shown. Core c_0 holds q_0, q_4 (adjacent to the port), and q_6, with six free positions; core c_1 holds eight qubits — the targets $\mathrm{q}_1, \mathrm{q}_2, \mathrm{q}_3$, the port-adjacent q_5, and four spectators — so its *only* free position is its port. The remaining circuit fragment is $g_1{=}\mathrm{CX}(\mathrm{q}_0, \mathrm{q}_1)$, $g_2{=}\mathrm{CX}(\mathrm{q}_0, \mathrm{q}_2)$, $g_3{=}\mathrm{CX}(\mathrm{q}_0, \mathrm{q}_3)$, $g_5{=}\mathrm{CX}(\mathrm{q}_0, \mathrm{q}_6)$, chained on q_0, plus an independent $g_4{=}\mathrm{CX}(\mathrm{q}_4, \mathrm{q}_5)$. The front is therefore $\{g_1, g_4\}$ — g_2 and g_3 share q_0 with g_1 and only surface after it executes. All front gates are adjacent-remote, so the plan-first stage fires.

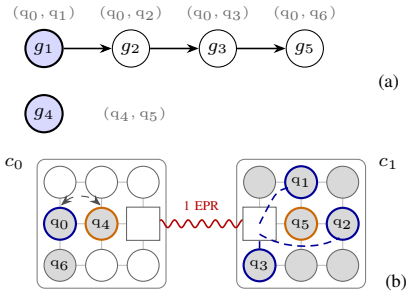


Fig. 4. Running example. (a) Remaining circuit fragment as a DAG; the front (shaded) is $\{g_1, g_4\}$. (b) Layout on two adjacent cores of architecture A. Blue circles are the committed TELEGATEPACKET's participants (q_0 and the targets $\mathrm{q}_1, \mathrm{q}_2, \mathrm{q}_3$, reached by cat-CXs, dashed); orange circles are the endpoints of the competing, already-ready single TELEGATE for g_4; grey circles are spectators. Core c_1's only free position is its port, so $\mathrm{free}(c_1) = 1 < f_{\min}$ and every TELEDATA into c_1 is capacity-blocked. The dashed double arrow is the packet's one preparation SWAP, which moves q_0 next to the port.

TABLE V
REMOTE PLANS GENERATED FOR THE FRONT OF FIGURE 4, WITH TOTAL GAINS (SECTION III-E).

Plan	Set	Gain	Readiness	Outcome
TELEGATEPACKET(q_0; $g_1 g_2 g_3$)	$\mathcal{P}$	3.0	1 prep SWAP	**committed**
TELEGATE(g_4)	$\mathcal{P}$	1.0	ready now	dominated
TELEGATE(g_1)	$\mathcal{P}$	1.0	1 prep SWAP	dominated
TELEDATA($\mathrm{q}_1 \to c_0$)	$\mathcal{P}$	1.0	prep needed	dominated
TELEDATA($\mathrm{q}_5 \to c_0$)	$\mathcal{P}$	1.0	prep needed	committed later
TELEDATA($\mathrm{q}_0 \to c_1$)	$\mathcal{B}$	≈2.1	blocked	margin $< m_{\text{assist}}$
TELEDATA($\mathrm{q}_4 \to c_1$)	$\mathcal{B}$	1.0	blocked	dominated

What PORTER does. Table V lists the generated plans. Seeding packet collection at g_1, the frontier simulation absorbs g_2 and g_3 (same control q_0, targets in c_1): a three-gate packet with direct gain 3. Both single telegates and all TELEDATA plans into c_0 gain ≈1. TELEDATA($\mathrm{q}_0 \to c_1$) would gain more — direct 1 for g_1, latent-direct 2 for g_2, g_3, minus 0.9 of lookahead for g_5, whose partner q_6 stays in c_0 — but c_1 fails the $f_{\min}$ test, so it lands in $\mathcal{B}$; its gain ≈2.1 does not exceed the packet's 3.0 by $m_{\text{assist}}{=}5$, so no capacity relief pre-empts. Plans are ranked by gain first — preparation cost is only a tie-break — so the *unready* packet (gain 3) beats the *ready* telegate for g_4 (gain 1). PORTER commits the packet, generates its one preparation SWAP (q_0 next to the port), then emits one cat-entangler, three cat-CXs (each target routed beside the port in turn), and the disentangler: *one* EPR pair for g_1, g_2, g_3. The packet's completion exposes g_4 and g_5 together; the locality-first stage drains the now-local g_5 first (one SWAP plus a local CX), and the remote stage then re-generates plans for g_4: the single telegate and TELEDATA($\mathrm{q}_5 \to c_0$) now tie at gain 1, and the capacity tie-break — negative for the TELEDATA, zero for the TELEGATE (Section III-E) — commits the teleport — q_5 lands on c_0's port and g_4 finishes as a local CX. Two EPR pairs for the whole fragment. This is not a stylised trace: it is exactly the operation stream the implementation emits on this instance, replayed and certified by the independent verifier of Appendix D, which includes the full operation-by-operation certified trace.

What finer-grained routers do here. A router that scores individual actions — SWAPs and inter-core operations in

one pool, as TELESABRE does — fires the zero-cost ready telegate for g_4 first. The deeper problem is the staging SWAP the packet needs: exchanging q_0 and q_4 moves g_1's control one step closer to the port while moving g_4's one step away, so under a front-averaged distance metric of the kind used by SABRE descendants its score is ≈ 0; the information that makes this SWAP worth applying — three remote CXs behind g_1 amortised onto one EPR pair — is a property of a *whole plan*, invisible at single-action granularity. That blind spot does not require a flat pool: DSABRE reaches its inter-core stage on this front (no front gate is intra-core) and scores hops, not plans, so the same amortisation is outside what its candidate can express.

Vocabulary compounds the gap. Gates g_2 and g_3 enter the front one at a time, so TELESABRE pays one telegate each, four EPR pairs for the fragment. DSABRE, restricted to TELEDATA, cannot move q_0 into the saturated c_1 at all — under its capacity floor a teleport into a core with fewer than two free slots is not a legal candidate — so it must first relay a free slot into c_1 from elsewhere, and ferrying q_1, q_2, q_3 out to c_0 instead costs three EPR pairs before g_4 is even addressed. Commitment finally protects the choice once made: only the packet's preparation SWAPs are generated — no staging effort is spent on the six losing plans — and the committed plan cannot be pre-empted mid-preparation by a candidate that its own staging transiently improved, a churn per-action re-scoring permits by construction.

APPENDIX G
INTER-CORE ACTION MIX

TABLE VI
INTER-CORE ACTIONS BY PRIMITIVE ON THE 64-QUBIT SUITE. EVERY ACTION CONSUMES EXACTLY ONE EPR PAIR, SO THE COLUMNS OF EACH ROUTER SUM TO ITS EPR COUNT. "PKT CX" IS THE NUMBER OF REMOTE CX GATES PORTER'S PACKETS EXECUTE — THE ONLY COLUMN IN WHICH AN ACTION COUNT AND A GATE COUNT DIFFER. PORTER'S COLUMN IS *one forward pass at the configured horizon*, THE SAME PROTOCOL AS EVERY COLUMN OF TABLE IV, WHICH THIS TABLE EXISTS TO SUPPORT; IT IS THEREFORE NOT TABLE II'S SELECTED ROUTE AND ITS EPR TOTALS DIFFER FROM THAT TABLE'S. TELESABRE DOES NOT CONVERGE ON RANDOM.

	TELESABRE		DSABRE	PORTER			
Circuit	TELEDATA	TELEGATE	TELEDATA	TELEDATA	TELEGATE	pkt	pkt CX
AE	259	94	165	147	0	28	65
GHZ	16	0	6	6	0	0	0
Graphstate	60	2	15	20	0	0	0
QFT	308	4	179	192	0	29	68
QNN	883	6	538	550	1	19	54
Random	—	—	732	432	228	77	157
QPEexact	281	8	187	177	3	42	130
QAOA	824	132	555	539	3	126	305
Multiplier	1558	43	1285	940	384	34	92
Total	**4189**	**289**	**3662**	**3003**	**619**	**355**	**871**

Table VI puts the three compilers' primitives side by side on the 64-qubit suite, which is the sharpest statement of what a larger action vocabulary is worth: every action listed costs exactly one EPR pair, so an action count and an EPR count are the same number — except for PORTER's packets, where one action executes several gates.

TELESABRE spends 289 of its 4,478 actions (6.5%) on single telegates, each serving exactly one remote CX, and uses the most EPR pairs of the three. DSABRE spends none. PORTER's cat-entangler actions are 1,438 of 4,500 (32.0%), but they are not the same object: 801 packets carry 1,976 remote CX gates, 2.47 per EPR pair, against 1.00 for the 637 single telegates. Packets are 56% of PORTER's cat-entangler actions and 76% of the gates those actions serve.

Two features of the table shape the ablation of Section V-C. First, the vocabulary is used very unevenly: Random and Multiplier account for 612 of PORTER's 637 single telegates, while QNN and QAOA account for 571 of its 801 packets. Second, amortisation and EPR consumption do not move together — PORTER spends more cat-entangler actions than TELESABRE and fewer EPR pairs than it, but also more EPR pairs than DSABRE, which spends none at all. Whether the extra vocabulary pays is therefore not answerable from this table; Section V-C answers it directly, and the answer is not the one the design assumed.

APPENDIX H
SUPPLEMENTARY TABLES

The tables below support claims made in Sections III and V of the main paper: the full parameter set (Table VII), the per-circuit heavy-hex and shared-layout rows summarised there (Tables VIII and IX), and local SWAP counts (Table XI). EPR pairs are the figure of merit throughout — a SWAP is a local operation costing no entanglement — which is why SWAP counts are reported here rather than in the main text.

TABLE VII
HARDWARE COST AND HEURISTIC PARAMETERS (SEE SECTION III).

Parameter	Symbol	Value
Min. free before TELEDATA	$f_{\min}$	2
Opportunistic-relief margin	m_{assist}	5.0
Local-SWAP extended-set weight	λ_{loc}	0.8
Local extended-set max layers	L_{loc}	12
Local extended-set max CX	N_{loc}	20
Packet collection window (layers)	–	120
Max packet size (remote CX)	–	8
TeleData lookahead max layers	L_{tp}	$\{4, 8, 16, 128\}^{\dagger}$
TeleData lookahead max CX	N_{tp}	$50^{\ddagger}$
Lookahead weight / decay / floor	$w_0, \gamma, w_{\min}$	0.9, 0.9, 0.5

† The teleport-lookahead horizon is not a fixed constant: PORTER routes the circuit once per candidate value and keeps the lowest-EPR result. The set is the same for every circuit; 16 is the configured value, routed first and winning ties (Section IV-D1).

‡ Inert at this value. The lookahead admits at most one gate on the moving qubit per simulated layer, so the collected count cannot exceed L_{tp} and the cap cannot bind while $N_{\text{tp}} \geq L_{\text{tp}}$; over 3 830–11 395 scoring calls per circuit it was never reached, and changing it leaves the emitted circuit bit-for-bit identical.

Table X gives the per-circuit EPR counts summarised by Table II of the main paper.

Table XI gives the intra-core SWAP counts for the runs of Table X, in the same row order. Both baselines and PORTER count intra-core SWAPs only; inter-core movement is charged in EPR pairs.

APPENDIX I
COMPUTATIONAL COMPLEXITY

Let N be the gate count, n the logical-qubit count, K the core count and M the per-core physical-qubit count.

TABLE VIII

ARCHITECTURE-INDEPENDENCE CHECK: THE SAME NINE-CIRCUIT 64-QUBIT SUITE ON FOUR IBM 27-QUBIT HEAVY-HEX CORES (108 PHYSICAL QUBITS), WIRED AS A *ring* (4 LINKS, EVERY CORE DEGREE 2) AND AS A *star* (3 LINKS, CORE 0 A HUB EVERY INTER-CORE ROUTE MUST CROSS). EPR PAIRS ONLY; THE QUESTION HERE IS ARCHITECTURAL SENSITIVITY, NOT THE EPR/SWAP TRADE-OFF OF TABLE II. THE RIGHT-HAND PAIR IS THE FIXED-LAYOUT CONTROL: PORTER ROUTING DSABRE'S OWN SABRELAYOUT CANDIDATES, SO THE TWO ROUTERS DIFFER ONLY IN THE ROUTING LOOP. REFINEMENT IS MEASURED ON THE MAIN SUITE ONLY (TABLE II), SO NO PORTER+R COLUMN APPEARS HERE. DSABRE REPRODUCES ITS OWN PUBLISHED NUMBERS FOR THESE TWO ARCHITECTURES ON ALL EIGHTEEN INSTANCES [3].

			own layout	DSABRE layout
Cores	Circuit	DSABRE	PORTER	PORTER
ring	ae	86	69	61
ring	ghz	2	2	2
ring	graphstate	12	12	13
ring	qft	110	109	110
ring	qnn	249	240	269
ring	random	465	465	419
ring	qpeexact	80	105	93
ring	qaoa	249	244	244
ring	multiplier	451	506	416
gmean (9)		85.2	86.1	**82.9**
star	ae	118	132	121
star	ghz	3	3	4
star	graphstate	10	16	11
star	qft	125	117	98
star	qnn	241	361	282
star	random	475	497	458
star	qpeexact	120	132	102
star	qaoa	251	384	243
star	multiplier	722	455	462
gmean (9)		101.1	113.3	**97.2**

TABLE IX

SHARED-LAYOUT COMPARISON, ALL SUITES: DSABRE AND PORTER ARE BOTH ROUTED FROM TELESABRE'S OWN INITIAL LAYOUT, EACH AT ITS OWN ROUTE BUDGET (SECTION IV-C). Δ_{TS} COLUMNS GIVE EACH ROUTER'S EPR CHANGE RELATIVE TO TELESABRE'S OWN-PROTOCOL RESULT; NEGATIVE IS BETTER. ROWS EXIST ONLY WHERE TELESABRE CONVERGES AND THUS PROVIDES A LAYOUT TO SHARE: ALL SIX CIRCUITS AT 25Q, EIGHT OF NINE AT 64Q (RANDOM MISSING), THREE OF NINE AT 80Q (GHZ, GRAPHSTATE, AND QPEEXACT); PER-SUITE GMEANS ARE OVER THE AVAILABLE ROWS, AND THE FINAL **ALL** ROW IS THE GMEAN OVER ALL 17.

		TELESABRE		DSABRE			PORTER		
Suite	Circuit	EPR	SWAP	EPR	SWAP	Δ_{TS}	EPR	$\mathrm{EPR}_{+\mathrm{R}}$	Δ_{TS}
25q	ae	119	346	66	310	−44.5	78	60	−49.6
25q	ghz	13	44	9	29	−30.8	10	8	−38.5
25q	graphstate	21	63	6	30	−71.4	14	6	−71.4
25q	qft	138	448	81	362	−41.3	100	97	−29.7
25q	qnn	515	820	145	685	−71.8	136	126	−75.5
25q	random	636	1367	419	1442	−34.1	403	389	−38.8
gmean (6)		106.6		51.0		**−52.2**	62.5	48.9	**−54.1**
64q	ae	353	1494	215	1672	−39.1	151	151	−57.2
64q	ghz	16	132	10	121	−37.5	10	8	−50.0
64q	graphstate	62	281	26	149	−58.1	46	24	−61.3
64q	qft	312	1711	223	1646	−28.5	290	257	−17.6
64q	qnn	889	5471	648	5707	−27.1	580	580	−34.8
64q	qpeexact	289	1706	193	1574	−33.2	186	186	−35.6
64q	qaoa	956	4189	673	4244	−29.6	598	598	−37.4
64q	multiplier	1601	9654	1564	9489	−2.3	873	755	−52.8
gmean (8)		284.5		189.2		**−33.5**	180.7	156.7	**−44.9**
80q	ghz	26	215	25	196	−3.8	19	12	−53.8
80q	graphstate	87	435	40	233	−54.0	64	32	−63.2
80q	qpeexact	784	2875	408	2732	−48.0	436	436	−44.4
gmean (3)		121.0		74.2		**−38.7**	80.9	55.1	**−54.5**
gmean, all (17)		173.0		100.9		**−41.7**	107.8	**86.4**	**−50.1**

Distance tables are precomputed once at construction in $O(KM^2 \log M + K^2 \log K)$, matching the two-level Dijkstra decomposition comparable compilers use [3]. Per iteration, classification and draining cost $O(|F|)$; intra-core SWAP scoring evaluates $O(M)$ candidate edges, each at $O(|F_{\mathrm{loc}}| + N_{\mathrm{loc}})$ over one DAG snapshot reused across candidates; remote-plan scoring evaluates $O(n)$ candidates at $O(N_{\mathrm{tp}})$ over the same snapshot, writing N_{tp} for the number of lookahead CX gates actually examined per candidate — at most the configured cap of 50, and in practice at most the horizon L_{tp}, since the collector admits one gate on the moving qubit per simulated layer (Appendix H). Free-slot queues are $O(1)$ amortised per layout mutation. Writing $C_{\mathrm{iter}} = O(|F| + M(|F_{\mathrm{loc}}| + N_{\mathrm{loc}}) + nN_{\mathrm{tp}})$ for that per-iteration cost, a route of I iterations costs $T_{\mathrm{pre}} + O(I \cdot C_{\mathrm{iter}})$. We keep I as an independent quantity: an iteration may apply a local or preparation SWAP without retiring a gate, and capacity relief adds further inter-core actions, so nothing here bounds I by the gate count N — the iteration and repeated-state guards of Section III-H bound it by construction instead. The shape is in line with other core-local distributed compilers [3], [4]. This is an operation count for *one* route, and it is not what the reported timings measure: those are complete protocol time over several routes (Section IV-C), so they neither validate this bound nor are bounded by it.

APPENDIX J
THE LAYOUT-DISCOUNT SEARCH (PORTER+S)

PORTER+S is a third operating protocol, reported here rather than in the main paper because it is a parameter search rather than a design decision, and because what it returns does not justify what it costs. It searches the one free scalar in the initial-layout heuristic — the per-layer discount γ of Section III-A, fixed at 0.99 everywhere else — in two phases.

Phase 1 builds a layout for each $\gamma \in \{0.80, 0.90, 0.95, 0.99, 1.0\}$ and routes it once at the configured horizon — five routes — keeping the lowest-EPR layout. Phase 2 runs the P=5 backward-refinement schedule of Section IV-D2 from two seeds: phase 1's winner, and the γ=0.99 layout when that is not already the winner. Refining the phase-1 winner alone is not reliably best, so both are run. Each seed selects a horizon before refining, so each costs $|\mathcal{H}| + P - 1 = 8$ routes as PORTER+R does, and the nominal budget is $5 + 8 = 13$ invocations of the routing loop when γ=0.99 wins phase 1 and $5 + 8 + 8 = 21$ otherwise. Each seed re-routes a layout at a horizon phase 1 already covered, so the number of *distinct* routes — which is what the released harness records — is 12 or 19, and falls below that where a per-pass guard stops a pass short; over the twenty-four instances it ranges from 10 to 19. Since γ=0.99 always competes and is refined from its selected route, PORTER+S cannot return a higher EPR count than PORTER or PORTER+R do.

No single γ dominates — all five win somewhere, 0.80 most

TABLE X
PER-CIRCUIT EPR COUNTS, OWN PROTOCOL — THE DETAIL BEHIND TABLE II OF THE MAIN PAPER. TELESABRE AND DSABRE REPORT THE BEST OF THREE SEEDS; PORTER ROUTES ONE DETERMINISTIC LAYOUT AT EACH OF FOUR CANDIDATE LOOKAHEAD HORIZONS; AND PORTER+R SPENDS EIGHT ROUTES OVER A FIVE-PASS BACKWARD-REFINEMENT SCHEDULE INSTEAD. Δ_{TS} IS THE EPR CHANGE OF WHICHEVER PORTER COLUMN IS STRONGER ON THAT ROW, AGAINST TELESABRE (%); NEGATIVE IS BETTER, AND THE BOLDED PORTER COLUMN IS THAT STRONGER ONE. *TELESABRE DOES NOT CONVERGE IN THREE SEEDS; EACH GMEAN ROW COVERS THE CIRCUITS IT COMPLETES. LOCAL SWAP COUNTS, IN THE SAME ROW ORDER, ARE IN TABLE XI.

Suite	Circuit	TELESABRE	DSABRE	PORTER	PORTER+R	Δ_{TS}
25q	ae	119	65	65	65	−45.4
25q	ghz	13	5	4	4	−69.2
25q	graphstate	21	6	7	4	−81.0
25q	qft	138	81	74	74	−46.4
25q	qnn	515	143	119	116	−77.5
25q	random	636	411	393	387	−39.2
gmean (6)		106.6	45.8	43.0	**38.9**	**−63.5**
64q	ae	353	165	157	157	−55.5
64q	ghz	16	6	6	6	−62.5
64q	graphstate	62	15	20	16	−74.2
64q	qft	312	179	221	221	−29.2
64q	qnn	889	538	550	512	−42.4
64q	random	*	732	732	715	—
64q	qpeexact	289	187	222	222	−23.2
64q	qaoa	956	555	668	668	−30.1
64q	multiplier	1601	1285	1076	1076	−32.8
gmean (8)		284.5	144.6	156.8	**151.1**	**−46.9**
80q	ae	*	332	265	265	—
80q	ghz	26	13	7	7	−73.1
80q	graphstate	87	20	28	24	−72.4
80q	qft	*	322	318	318	—
80q	qnn	*	807	1072	887	—
80q	random	*	10551	10926	10848	—
80q	qpeexact	784	347	276	276	−64.8
80q	qaoa	*	862	1202	889	—
80q	multiplier	*	4841	2874	2130	—
gmean (3)		121.0	44.9	37.8	**35.9**	**−70.3**

TABLE XI
LOCAL SWAP COUNTS, OWN PROTOCOL. ROW ORDER AND THE * NON-CONVERGENCE MARKER MATCH TABLE X.

Suite	Circuit	TELESABRE	DSABRE	PORTER	PORTER+R
25q	ae	346	309	328	328
25q	ghz	44	17	29	29
25q	graphstate	63	31	34	19
25q	qft	448	365	327	327
25q	qnn	820	711	642	603
25q	random	1367	1448	1342	1333
64q	ae	1494	1325	1468	1468
64q	ghz	132	39	86	86
64q	graphstate	281	88	131	103
64q	qft	1711	1480	1805	1805
64q	qnn	5471	5260	5336	5503
64q	random	*	3603	3931	4324
64q	qpeexact	1706	1576	1729	1729
64q	qaoa	4189	3734	4657	4657
64q	multiplier	9654	8969	8649	8649
80q	ae	*	2624	2311	2311
80q	ghz	215	80	115	115
80q	graphstate	435	97	198	150
80q	qft	*	2362	2572	2572
80q	qnn	*	8445	9669	9243
80q	random	*	56624	62038	62138
80q	qpeexact	2875	2254	2308	2308
80q	qaoa	*	6067	7857	6519
80q	multiplier	*	22029	16914	14678

often (nine of twenty-four instances) — which is why the value is searched rather than retuned. The search is deterministic and takes no seed.

What it buys. Table XII gives the result. Pooled over the twenty-four instances PORTER+S reaches 152.6 EPR pairs against PORTER+R's 156.7, a 2.6% reduction, and 0.89× DSABRE's 171.9 against PORTER+R's 0.91×. It improves on eight instances, ties on sixteen and — being a superset schedule — loses on none. Its one qualitative effect is at 64 qubits, the single suite PORTER loses under its own protocol: the ratio against DSABRE falls from PORTER+R's 1.04× to 0.99×, turning the one suite it trails on into a narrow win.

What it costs. The same twenty-four instances take 796 s against PORTER+R's 386 s — 2.1× the wall clock, and 4× DSABRE's complete protocol cost of 182 s. The expenditure is badly distributed: the 80-qubit suite consumes 592 of those seconds and returns 0.5%, and 80q AE alone spends 40.0 s against PORTER+R's 11.1 s — a factor of four — to return 277 pairs, the figure PORTER+R already reaches. On three quarters of the suite the search returns nothing at all.

Why it is not in the main paper. Three reasons beyond the return. It cannot appear in the layout-against-router grid of Table III, since its mechanism *is* choosing the initial layout and a supplied layout would destroy that control — so it can only ever be a fourth own-protocol column. It introduces a second in-sample-fitted quantity, the γ set, alongside the horizon candidate sets whose in-sample fitting Section IV-D1 already declares. And at 12–19 routing-loop invocations and 6.55× DSABRE's compile time it sits on the wrong side of the argument Section V-E makes, that PORTER's time advantage lies in the number of routes it spends rather than their individual price. We report it here so the shape of the EPR/time frontier past PORTER+R is on the record.

PORTER+SR is subsumed. An earlier version of this work also reported PORTER+SR, the better of horizon selection and the γ-search. Phase 2 refines from the γ=0.99 layout among its seeds, and that is the layout PORTER builds by default, so once refinement runs on top of horizon selection the search contains the selected route and cannot lose to it. PORTER+SR does not differ from PORTER+S on any of the twenty-four instances, at 19% more compile time, and is retired.

APPENDIX K
THREATS TO VALIDITY

This appendix collects the assumptions and limitations the evaluation of the main paper rests on. Each is stated where it first matters; they are gathered here so a reader can weigh them together.

Mechanism attribution. No experiment replaces a plan with a hop (Section III-D), so neither the plan abstraction nor the commitment rule is isolated, and the reductions of Section V-A belong to the whole system. Separating their share needs a hop-granular candidate set, which is a different router.

Completion is empirical. PORTER guarantees correctness and termination by construction but not completion (Section VII). Every instance completes under the selected-horizon

TABLE XII
PORTER+S AGAINST PORTER AND PORTER+R ON THE MAIN TWENTY-FOUR INSTANCES, OWN PROTOCOL. γ^* IS THE DISCOUNT PHASE 1 SELECTED; t IS COMPLETE PROTOCOL TIME IN SECONDS. Δ IS PORTER+S AGAINST PORTER+R AND IS DASHED WHERE THE TWO COINCIDE. PORTER+S IS NEVER WORSE, BY CONSTRUCTION. LOWER IS BETTER.

		EPR					t (s)	
Suite	Circuit	PORTER	+R	+S	Δ %	γ^*	+R	+S
25q	ae	65	65	62	−4.6	0.80	0.8	1.7
25q	ghz	4	4	4	—	0.80	0.0	0.0
25q	graphstate	7	4	4	—	0.80	0.0	0.1
25q	qft	74	74	74	—	0.99	0.9	1.4
25q	qnn	119	116	116	—	0.80	2.1	4.8
25q	random	393	387	387	—	0.99	4.0	6.2
gmean (6)		43.0	38.9	**38.6**	**−0.8**		0.36	0.77
64q	ae	157	157	157	—	0.99	6.7	8.3
64q	ghz	6	6	6	—	0.80	0.0	0.1
64q	graphstate	20	16	16	—	0.80	0.4	1.0
64q	qft	221	221	221	—	0.99	4.3	9.0
64q	qnn	550	512	512	—	0.80	23.7	59.5
64q	random	732	715	664	−7.1	0.90	17.4	49.4
64q	qpeexact	222	222	196	−11.7	0.90	4.1	10.2
64q	qaoa	668	668	574	−14.1	0.95	22.0	52.2
64q	multiplier	1076	1076	998	−7.2	0.90	14.9	36.0
gmean (9)		186.1	179.6	**171.3**	**−4.6**		3.93	9.23
80q	ae	265	265	262	−1.1	0.90	11.8	30.3
80q	ghz	7	7	7	—	0.80	0.1	0.1
80q	graphstate	28	24	24	—	0.80	0.7	2.0
80q	qft	318	318	275	−13.5	1.00	5.0	11.9
80q	qnn	1072	887	887	—	0.80	48.6	114.0
80q	random	10926	10848	10848	—	0.90	214.2	699.5
80q	qpeexact	276	276	274	−0.7	1.00	18.3	33.1
80q	qaoa	1202	889	889	—	1.00	47.9	111.5
80q	multiplier	2874	2130	2130	—	0.90	28.5	61.5
gmean (9)		384.7	346.1	**339.8**	**−1.8**		9.35	23.07
gmean, all (24)		169.4	156.7	**152.6**	**−2.6**		3.00	7.00

protocol; at a fixed horizon one does not, so that record belongs to the protocol and this benchmark rather than to the router.

Benchmark coverage. Nine MQT-Bench families at three widths, but the scale-up series is two families only — nothing supports a claim about scaling on irregular circuits at those widths — and the heavy-hex study is two core graphs, not a survey.

Parameters fitted in-sample. The sweep behind the horizon portfolio, and the four values taken from it, were chosen by inspecting these same instances (Section IV-D1), so the *magnitude* of that gain is in-sample; the portfolio construction bounds the downside, but no out-of-sample validation has been performed. The same caution applies in weaker form to $f_{\min}$, m_{assist} and the packet window, fixed early and not swept.

Reconstructed benchmark instances. The 80-qubit AE and QNN circuits were reconstructed rather than generated (Appendix A). We release the scripts and validate QNN against the 64-qubit file, but they remain our circuits, not the generator's.

Patched baseline binary. All TELESABRE numbers use the patched binary of Section IV-B. Beyond the bound stated there — it moves only qubits the stock binary leaves unplaced, and architecture H is unchanged — we have not audited its effect.

Unequal native budgets. The own-protocol comparison gives DSABRE nine routes to PORTER's four. That is each compiler's own convention, not a controlled budget — hence the shared-layout experiment, and hence compile time throughout.

Aggregation. Geometric means are taken over jointly completed instances, so a baseline failing hard instances is scored only on easier ones; completion counts accompany every such mean. Being unweighted, they also let the two cheapest circuit families move a suite figure out of proportion to their cost — which is why Table III reports both bases.

Implementation and timing. PORTER and DSABRE are Python and TELESABRE is C, so per-route times compare implementations as much as algorithms, and all are wall-clock on one machine. Times from different measurement sessions are not comparable — the same protocol on this machine has measured 1.7× apart between sessions — so every timing column reported here comes from a single run. *Results* do not depend on load: horizon selection budgets a speculative route in DAG layers peeled rather than seconds (Section IV-B).

Cost model. One EPR pair per inter-core action, independent of hop count and generation protocol; link contention, entanglement latency and fidelity, purification and classical communication are outside the model (Section II-B). Conclusions are relative to it.

REFERENCES

[1] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking software and design automation tools for quantum computing," *Quantum*, vol. 7, p. 1062, 2023.

[2] P. Andres-Martinez, D. Mills, T. Forrer, and L. Henaut, "CQCL/pytket-dqc," https://github.com/CQCL/pytket-dqc, Jun. 2024.

[3] S. Li, "dSABRE: A SABRE-style router for multi-core distributed quantum computers," 2026, manuscript, Centre for Quantum Software and Information, University of Technology Sydney. Code: https://github.com/ebony72/dsabre.

[4] E. Russo, E. Vinciguerra, M. Palesi, D. Patti, G. Ascia, and V. Catania, "TeleSABRE: Heuristic layout synthesis in multi-core quantum systems with teleport interconnect," in *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2025, pp. 749–758, arXiv:2505.08928. Code: https://github.com/Haimrich/telesabre.

[5] A. Yimsiriwattana and S. J. Lomonaco Jr., "Generalized GHZ states and distributed quantum computing," *arXiv preprint quant-ph/0402148*, 2004.